

Liquidación semanal Empresa de mensajería

Programación III
Parcial 1
Armenia 1 de octubre de 2026
Alejandro Vargas Rodríguez
Juan Pablo Rios
Julián E. Gutiérrez Posada
Ingeniería de Sistemas y Computación



UNIVERSIDAD
DEL QUINDÍO

Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN



UNIQUINDÍO
en conexión territorial

www.uniquindio.edu.co

Presentación del proyecto

En este trabajo presentamos el programa de liquidación semanal de una empresa de mensajería para el primer parcial de Programación III. El programa revisa los servicios, calcula el pago de los repartidores y muestra los reportes R1 a R8. También permite ingresar un servicio adicional y consultar un comprobante individual.

Con los datos base obtuvimos 108 servicios válidos y 10 rechazados. La empresa recorrió 4.009,50 km y el total neto a pagar fue de \$9.375.562,50. Estos resultados se pueden comprobar en la salida completa que incluimos al final del informe.

Objetivo y alcance

Nuestro objetivo fue organizar una solución que pudiéramos ejecutar desde la consola y revisar por módulos. Usamos listas, mapas, tuplas, funciones de Enum y comprehensions. El programa no usa recursividad propia, structs, lectura o escritura con File, procesos, Mix, librerías externas ni try/rescue para validar.

Partimos de nuestro conocimiento y con la ayuda de la IA nos apoyamos. Después incorporamos sus módulos al repositorio, revisamos su funcionamiento y ejecutamos las pruebas que aparecen en este documento.

Organización del informe

La Parte A explica las colecciones, las estructuras de control y la separación de responsabilidades. Luego mostramos las reglas de liquidación y los resultados principales de la Parte B. La explicación de R6 desarrolla el promedio ponderado. La Parte C contiene el ranking configurable, la combinación de mapas y las mediciones.

Repositorio y ejecución

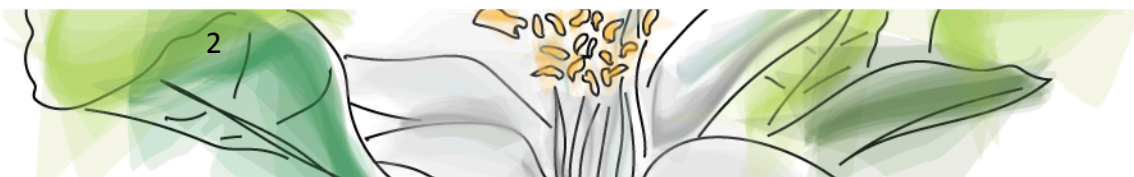
Repositorio del grupo: <https://github.com/Reposao/parcial-elixir-mensajeria>

Integrantes: Alejandro Vargas Rodríguez (Reposao) y Juan Pablo Ríos (Juanpa0031). La ejecución registrada se realizó en Windows, desde PowerShell en Visual Studio Code. En las pruebas de IEx se mostró Elixir 1.19.5.

Windows: .\ejecutar.bat

Linux o macOS: bash [ejecutar.sh](#)

Se necesita Elixir disponible en la terminal. Al iniciar, Enter omite el servicio adicional. Los decimales se ingresan con punto. Después de los reportes se escribe un código, por ejemplo M03, para consultar el comprobante.



Parte A Diseño de los datos

Por qué usamos listas y mapas

Los datos llegan como listas, por lo que mantuvimos una lista de repartidores, otra de zonas y otra de servicios. Así podemos recorrer cada conjunto con Enum.map, Enum.filter o for. Cada elemento es un mapa con claves que indican qué representa cada valor; por ejemplo, servicio.kilometros es más claro que recordar la posición de ese dato en una lista.

Colección	Campos del mapa
Repartidores	codigo, nombre, bicicleta
Zonas	id, nombre, area
Servicios	repartidor, zona, día, kilometros, retraso
Liquidaciones	codigo, nombre, kilometros, valor_servicios, bonificaciones, alquiler, neto, detalle

Para R3 usamos un mapa con el día como clave y los kilómetros como valor. Esta representación permite consultar un día directamente y combinar los resultados con el mapa de la empresa aliada. En los rechazos guardamos un mapa con servicio y motivo, de modo que no se pierde el registro original.

Tuplas para comunicar resultados

La validación devuelve { :ok, servicio } o { :error, motivo }. Esto permite distinguir una revisión correcta de un rechazo sin usar excepciones. La conversión de la entrada utiliza las mismas etiquetas y añade { :ok, :omitido } cuando el usuario presiona Enter. separar/3 devuelve { validos, rechazados }, dos listas que después se usan con propósitos diferentes.

Datos del grupo

Elemento	Cantidad o condición
Repartidores	10; cinco tienen bicicleta de la empresa
Zonas	4: Centro, Norte, Sur y Occidente
Servicios base	118 registros: 108 válidos y 10 rechazados
Días	Servicios válidos en los días 1 a 6
Motivos de rechazo	Dos registros por cada uno de los cinco motivos
Repartidor sin actividad	M10 no tiene servicios; su liquidación es cero

Los nueve repartidores activos tienen los mismos kilómetros diarios para comprobar que R5 conserva todos los empates. Sus retrasos son distintos y por eso los pagos y los promedios de puntualidad cambian. M10 permite revisar que un repartidor sin actividad no desaparezca del reporte. Aunque figura con bicicleta, no se le cobra alquiler porque no trabajó.

Parte A Módulos y recorrido del programa

Archivo y módulo	Responsabilidad
datos.exs · Datos	Entrega las listas de repartidores, zonas y servicios.
util2.exs · Util2	Lee texto, muestra mensajes y presenta decimales.
validacion.exs · Validacion	Convierte la entrada y valida en el orden exigido.
liquidacion.exs · Liquidacion	Calcula valores, detalles diarios, liquidaciones y datos de los reportes.
investigacion.exs · Investigacion	Configura rankings, combina mapas y mide dos alternativas de R3.
reportes.exs · Reportes	Imprime R1 a R8, comprobantes y resultados de investigación.
principal.exs · Mensajería	Coordina los módulos y ejecuta main/0.

Funciones puras e impuras

Validacion.validar/3 y Liquidacion.valor_servicio/1 son puras: dependen de los argumentos y devuelven un resultado sin leer la consola ni imprimir. Lo mismo ocurre con las funciones de cálculo de Liquidacion, ranking/2, combinar/2 y kilometros_con_reduce/1. Datos entrega listas constantes y Util2.decimal/1 solo transforma un valor en texto.

Util2.ingresar/2 es impura porque lee la consola. Util2.mostrar/2 y las funciones que imprimen en Reportes también lo son. Mensajeria.main/0 coordina esa interacción. Investigacion.medir/1 tampoco es pura: observa el tiempo de ejecución y puede devolver tiempos distintos con los mismos servicios. Separar estos casos permite probar los cálculos sin depender de lo que escriba el usuario.

Estructuras de control

Usamos with para encadenar las cinco validaciones y detenernos en el primer error. case distingue resultados de conversión, búsquedas y validación; cond selecciona el factor de puntualidad. if resuelve condiciones simples como el bono y el alquiler. Las guardas comprueban los tipos y rangos antes de aceptar un número.

Enum.map transforma cada elemento, Enum.filter selecciona registros y Enum.reduce construye acumulados. for expresa recorridos y filtros, como los días trabajados. Enum.all? y Enum.any? permiten revisar cobertura de zonas y cumplimiento de metas. No definimos funciones que se llamen a sí mismas.

Secuencia de ejecución

Primero se cargan y separan los datos base. Después se pregunta una sola vez por el servicio adicional. Con los servicios válidos definitivos se calculan las liquidaciones y se imprimen los reportes. Finalmente se consulta el comprobante y se muestran las investigaciones. Los rechazados se conservan para R1 y no entran en las sumas.

Parte B Validación y liquidación

Orden de validación

Paso	Regla	Motivo al fallar
1	Repartidor existente	repartidor_desconocido
2	Zona existente	zona_desconocida
3	Día entero entre 1 y 6	dia_invalido
4	Kilómetros numéricos mayores que 0 y hasta 45	kilometros_fuera_de_rango
5	Retraso numérico entre -30 y 180	retraso_invalido

El registro con M98, Z99, día 7, 0 km y retraso 181 tiene varios errores. Se rechaza por repartidor_desconocido porque esa es la primera regla que falla. Con este caso comprobamos que el programa no cambia el motivo según un error posterior.

La entrada adicional se separa por punto y coma y debe tener cinco campos. Integer.parse y Float.parse solo se aceptan si no queda texto sobrante. Convertir un número no significa que esté dentro del rango: primero se convierte y después se valida. Un formato inválido recibe su mensaje; un registro convertible que falla una regla se conserva en R1.

Reglas de pago

Retraso del servicio	Factor sobre km × \$2.500
Hasta 0 minutos	1,08: aumento del 8 %
Mayor que 0 y hasta 10	1,00: sin ajuste
Mayor que 10 y hasta 30	0,90: descuento del 10 %
Mayor que 30	0,75: descuento del 25 %

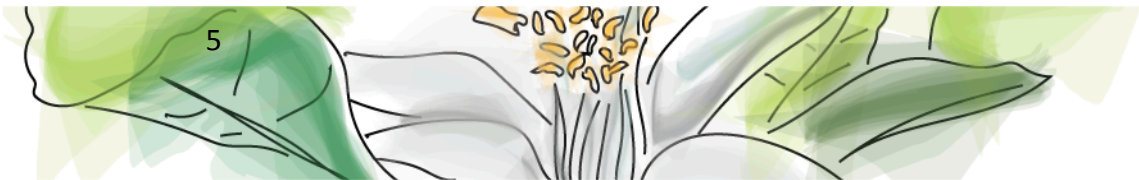
Se suma el valor ajustado de los servicios de cada repartidor y día. Si alcanza 80 km válidos, recibe \$15.000 de productividad una sola vez ese día. El alquiler es de \$10.000 por día con al menos un servicio válido y solo se cobra a quien tiene bicicleta de la empresa.

neto = valor_servicios + bonificaciones - alquiler

La bonificación del 8 % pertenece al valor de cada servicio puntual; la de \$15.000 depende de los kilómetros diarios. Son conceptos distintos y no se deben aplicar como si fueran el mismo bono.

Ejemplo comprobado

Un servicio de 10 km con retraso -5 vale $10 \times 2.500 \times 1,08 = \27.000 . Para M03, la ejecución base suma \$949.937,50 en servicios, \$60.000 en bonos y \$60.000 de alquiler. Su neto es \$949.937,50, igual en R4 y en el comprobante. Los días 1, 2, 3 y 5 reciben bono; los días 4 y 6 no alcanzan 80 km.



Parte B Resultados de los reportes

La ejecución del anexo omite el servicio adicional y consulta M03. Por eso todos los valores de esta página corresponden únicamente a los datos base.

Reporte	Resultado y comprobación
R1 Rechazados	10 registros; dos por cada motivo. Se muestra el registro completo y el primer error.
R2 Densidad	Orden: Z1, Z3, Z2, Z4. Z1: 1.017 km / 6,5 km² = 156,46 km/km².
R3 Meta diaria	Se alcanza la meta de 500 km los días 1 a 5; el día 6 suma 360 km. Todos los días: false. Al menos un día: true.
R4 Liquidación	Incluye los diez repartidores. M09 encabeza con \$1.196.750,00. M10 aparece con todos los valores en cero.
R5 Primeros por día	M01 a M09 empatan cada día. Todos acumulan seis primeros lugares.
R6 Puntualidad	M09 tiene el menor retraso ponderado: 11,45 min y 12 servicios válidos.
R7 Totales	Neto total \$9.375.562,50; 4.009,50 km; costo por km \$2.338,34.
R8 Cobertura	M01 a M09 tienen al menos un servicio válido en las cuatro zonas; M10 no cumple.

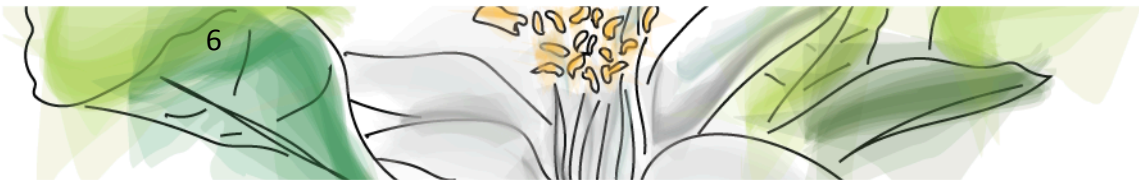
Kilómetros diarios y meta

Día	Kilómetros	Meta de 500 km
1	720,00	Sí
2	756,00	Sí
3	720,00	Sí
4	711,00	Sí
5	742,50	Sí
6	360,00	No

Decisiones para casos sin actividad y empates

En R2 aparecen todas las zonas, aunque alguna no tenga servicios. En R3 siempre aparecen los seis días, con cero cuando no hay actividad. En R4 se mantienen todos los repartidores. Para R5, un día sin kilómetros no asigna primer lugar; cuando sí hay actividad, se incluyen todos los empatados.

En R6 se devuelven todos los repartidores que empatan en el menor promedio ponderado. Si ninguno tiene tres servicios válidos, se informa que no hay elegibles. En R7, cuando no hay kilómetros, el promedio se presenta como cero para evitar una división por cero. Estas decisiones están documentadas en el código.



Explicación de R6 Puntualidad ponderada

R6 compara a los repartidores que tienen por lo menos tres servicios válidos en toda la semana. El menor retraso ponderado es el mejor resultado. Los minutos negativos se conservan porque representan entregas anticipadas.

Promedio simple = suma(retrasos) / cantidad de servicios
Promedio ponderado = suma(retraso × km) / suma(km)

El promedio simple da el mismo peso a cada entrega. El ponderado considera la distancia: un retraso en un servicio largo influye más que el mismo retraso en un recorrido corto. Por eso usamos los kilómetros como peso, tal como pide el enunciado.

Ejemplo con los doce servicios de M03

Día	Km de cada servicio	Retrasos en minutos
1	40 y 40	10 y 11
2	45 y 39	11 y 20
3	38,5 y 41,5	20 y 30
4	35 y 44	30 y 31
5	42 y 40,5	31 y 60
6	18 y 22	60 y 180

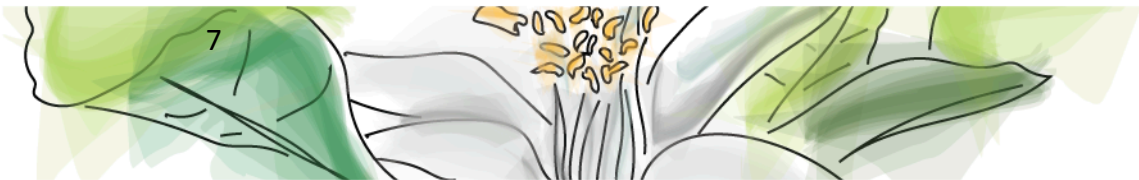
La suma de los retrasos de M03 es 494 minutos. El promedio simple es $494 / 12 = 41,1667$ minutos, que se muestra como 41,17. La suma de retraso por kilómetros es 15.316; al dividirla entre 445,5 km obtenemos 34,3793 minutos, que se muestra como 34,38.

La entrega con 180 minutos de retraso recorrió 22 km, una distancia menor que la de la mayoría de sus servicios. En el promedio simple pesa igual que una entrega de 45 km; en el ponderado pesa según sus 22 km. Eso ayuda a entender por qué los dos resultados de M03 son distintos. No se borra ese retraso: participa con la distancia que le corresponde.

Comparación con los datos base

Código	Simple en min	Ponderado en min
M01	14,92	13,53
M02	22,25	20,35
M03	41,17	34,38
M04	54,63	50,69
M05	49,75	56,81
M06	42,67	46,25
M07	37,17	39,53
M08	29,83	34,20
M09	10,92	11,45

M09 tiene el menor ponderado y aparece como ganador de R6. M10 queda fuera porque no tiene servicios. La diferencia entre M07 y M08 muestra que elegir el promedio simple también puede cambiar el orden de comparación: M08 tiene menor simple, pero M07 tiene menor ponderado.



Parte C Ranking con keyword lists

Qué investigamos

Una keyword list es una lista de tuplas de dos elementos cuya clave es un átomo. La escritura [campo: :kilometros, orden: :desc] equivale a [{:campo, :kilometros}, {:orden, :desc}]. La usamos para pasar opciones con nombres claros a ranking/2, sin crear una función diferente para cada criterio [1].

Las listas conservan el orden de sus elementos y pueden contener claves repetidas. En nuestro uso cada opción se pasa una sola vez. Keyword.get/3 permite leerla y definir un valor predeterminado cuando no aparece. Los mapas siguen siendo más apropiados para los registros; aquí la keyword list describe una configuración corta.

Implementación del proyecto

```
def ranking(coleccion, opciones) do
  campo = Keyword.get(opciones, :campo, :neto)
  orden = Keyword.get(opciones, :orden, :desc)
  Enum.sort_by(coleccion, fn dato -> dato[campo] end, orden)
end
```

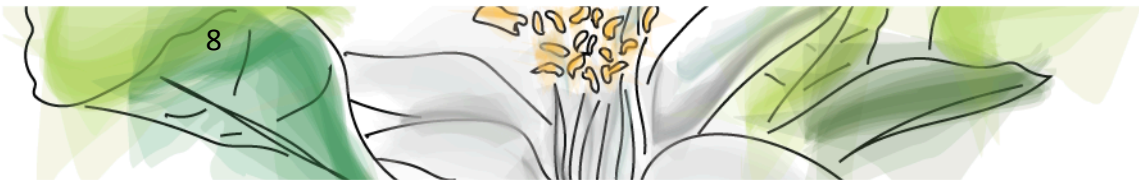
La función devuelve una nueva lista ordenada. Si la keyword list está vacía, usa neto de mayor a menor. campo selecciona la clave del mapa y orden admite :asc o :desc. En este proyecto la función recibe mapas con el campo solicitado y opciones controladas por el programa; no es un validador general de opciones introducidas por el usuario.

Llamada	Comportamiento
ranking(liquidaciones, [])	Ordena por neto descendente.
ranking(liquidaciones, campo: :kilometros, orden: :desc)	Ordena por kilómetros de mayor a menor.
ranking(liquidaciones, campo: :neto, orden: :asc)	Permite consultar primero los netos menores.

Cuando la keyword list es el último argumento, Elixir permite omitir sus corchetes. La segunda llamada de la tabla sigue pasando dos argumentos: la colección y la lista de opciones [1].

Resultado registrado

La salida completa muestra el ranking por kilómetros descendentes. M01 a M09 tienen 445,50 km y M10 tiene 0. El empate entre los activos no indica que ranking/2 esté fallando: la opción compara kilómetros, no el neto. R4 sí muestra diferencias porque ordena los pagos semanales.



Parte C Combinación de mapas y mediciones

Map.merge con dos y tres argumentos

El mapa propio sale de R3. Al combinarlo con el aliado, Map.merge/2 conserva las claves de ambos, pero en una coincidencia toma el valor del segundo mapa. Eso reemplaza kilómetros y no resuelve la suma de las dos empresas. Map.merge/3 permite definir cómo resolver cada coincidencia [3].

Map.merge(propio, aliada, fn_día, km1, km2 -> km1 + km2 end)

Día	Propio	Aliado	merge/2	merge/3
1	720	580,5	580,5	1.300,5
2	756	430	430	1.186
3	720	510	510	1.230
4	711	Sin clave	711	711
5	742,5	625	625	1.367,5
6	360	Sin clave	360	360
7	Sin clave	180	180	180

El día 7 se conserva con 180 km porque existe en el mapa aliado. Esto no cambia el rango 1..6 de nuestra empresa: la combinación representa información de las dos empresas y no redefine los días de liquidación.

Medición propia con timer

Comparamos dos formas de producir el mapa diario de R3 con los 108 servicios válidos. La primera filtra los servicios para cada uno de los seis días. La segunda prepara un mapa con esos días en cero y acumula cada servicio con Enum.reduce. Cada alternativa se ejecuta 100 veces dentro de :timer.tc/1; la impresión ocurre después. Esta función devuelve el tiempo en microsegundos y el resultado calculado [4].

Alternativa	100 repeticiones	Promedio derivado
Filtros por día	1.024 µs	10,24 µs por repetición
Acumulación con reduce	409 µs	4,09 µs por repetición

Estos valores pertenecen a la ejecución guardada en salida-programa.txt; no son tiempos garantizados. El cociente 1.024 / 409 es aproximadamente 2,50. En esa ejecución reduce tardó menos y la comparación de las listas de resultados devolvió true.

Parte B Pruebas e interacción

Comprobaciones registradas

Caso	Resultado observado
Separación de datos base	length(validos) = 108 y length(rechazados) = 10.
Conteo de motivos	Enum.frequencies_by mostró dos rechazos por cada motivo.
Valor de un servicio	10 km y retraso -5 produjeron 27000.0.
Repartidor sin servicios	M10 apareció con kilómetros, alquiler, bonos y neto en cero.
Comprobante M03	Su neto fue \$949.937,50, igual al de R4.
Código NOEXISTE	Se informó que no existe un repartidor con ese código.
Entrada M03;Z2;4;hola;-3	Se informó formato_invalido y el programa siguió con R1.
Omisión con Enter	No se agregó servicio; se generaron los reportes base.
Comparación de R3	Las dos alternativas produjeron resultados iguales.

Servicio adicional válido

También se ejecutó el programa con M03;Z2;4;22.5;-3. La entrada agregó 22,5 km al día 4. El total de ese día pasó de 711 a 733,5 km, y los kilómetros semanales de M03 pasaron de 445,5 a 468. Sus promedios cambiaron a 37,77 minutos simples y 32,58 ponderados, como se vio en la ejecución.

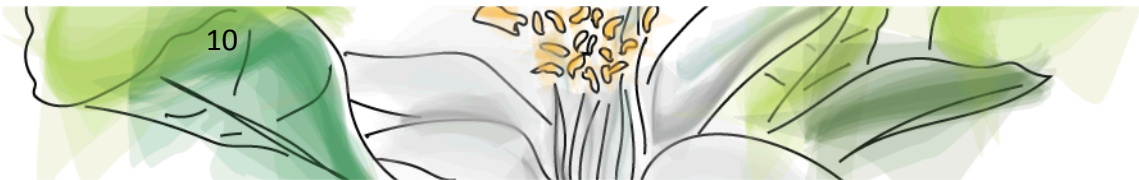
El servicio vale $22,5 \times 2.500 \times 1,08 = \60.750 . M03 pasa de 79 a 101,5 km en el día 4, por lo que recibe un bono adicional de \$15.000. El alquiler de ese día no aumenta: ya tenía servicios válidos. El neto esperado, derivado de esas reglas, es \$1.025.687,50 y el total empresarial esperado es \$9.451.312,50.

Límites y cobertura de los datos

La lista base contiene 0 y 45,1 km para rechazo, y 45 km como valor admitido. Incluye retrasos -30 y 180 dentro del rango, además de -30,1 y 180,1 fuera de él. También contiene los límites 0, 10 y 30 de puntualidad. Los 79 km del día 4 no generan bono; los 80 km de los días 1 y 3 sí lo generan.

Salida completa

El anexo reproduce la ejecución guardada por el grupo, sin resumir R1 a R8. Incluye la omisión del servicio adicional, el comprobante M03, el ranking, los mapas y los tiempos de investigación. El archivo también quedó registrado en el repositorio.



Parte D Bitácora

Aportes registrados en GitHub

Integrante	Commits de integración que permiten ubicar sus aportes
Juan Pablo Ríos	840acc1 datos; 986027b liquidación; c0fe446 reportes R1 a R8 y comprobantes.
Alejandro Vargas Rodríguez	73fddd9 Util2; defdcfc validación; ed55239 investigación; 992b33b principal y ejecutables; b9c6c4e saltos de línea; a0c6ece README; 6f3cd17 salida de prueba.

La asistencia fue útil para proponer una organización inicial y para resolver problemas de ejecución. Aun así, un archivo generado no prueba por sí solo que sus resultados sean correctos. Por eso dejamos resultados de consola y ejemplos numéricos que se pueden volver a ejecutar y contrastar.

Parte D Caso de error y reflexión

Caso de error durante la carga de Validacion

Al ejecutar `Validacion.separar/3` en IEx apareció `UndefinedFunctionError` con el mensaje de que el módulo `Validacion` no estaba disponible. La llamada se había escrito correctamente; el problema estaba en que el módulo todavía no se había cargado en esa sesión. Al intentar `c("validacion.exs")`, la consola devolvió `[]`, sin indicar un módulo compilado.

La corrección fue revisar que el archivo tuviera el código y estuviera guardado, volver a cargarlo y repetir la llamada. Después se obtuvo la separación esperada: 108 válidos, 10 rechazados y dos registros por motivo. La comprobación importante fue el resultado final, no solamente que dejara de aparecer el mensaje de error.

```
{validos, rechazados} = Validacion.separar(
  Datos.servicios(), Datos.repartidores(), Datos.zonas()
)
length(validos)      # 108
length(rechazados)   # 10
```

Reflexión

Lo que más nos mostró el proceso fue la necesidad de distinguir el código, la sesión de Elixir y el repositorio. Un error de carga no se soluciona cambiando una fórmula, y un comando de Git no pertenece a IEx. Revisar dónde estábamos ejecutando cada instrucción evitó seguir cambiando archivos que no eran el problema.

La IA nos permitió avanzar con el trabajo y acompañó la revisión, pero también hizo necesario leer el alcance con cuidado. Por ejemplo, el `Util2` de clase tenía reintentos recursivos y no podía copiarse completo. En el proyecto se utiliza una lectura por intento y los errores se comunican con tuplas y mensajes.

Conclusiones y referencias

Con la ejecución base pudimos comprobar que los rechazos se separan antes de liquidar y que los reportes utilizan los servicios válidos. El comprobante de M03 coincide con R4 y M10 conserva su liquidación en cero. La meta empresarial se cumple en cinco días, pero no en el sexto.

La separación modular permite ubicar cada regla y revisar sus efectos. Validacion concentra los motivos de rechazo; Liquidacion calcula los valores; Reportes presenta los resultados. La investigación reutiliza esos datos sin alterar la liquidación de la empresa propia.

Referencias utilizadas

Gutiérrez Posada, J. E. Enunciado del Parcial 1 Liquidación semanal de una empresa de mensajería. Programación III, Universidad del Quindío. Archivo suministrado en el curso: N-G2 - Jugutier - Empresa mensajería.pdf.

[1] Elixir. Documentación oficial de Keyword. Definición de keyword lists, sintaxis y Keyword.get/3. <https://hexdocs.pm/elixir/Keyword.html>

[2] Elixir. Documentación oficial de Enum. Ordenamiento mediante Enum.sort_by/3 y recorridos de colecciones. <https://hexdocs.pm/elixir/Enum.html>

[3] Elixir. Documentación oficial de Map. Comportamiento de Map.merge/2 y Map.merge/3. <https://hexdocs.pm/elixir/Map.html>

[4] Erlang/OTP. Documentación oficial de timer. Medición con tc/1 y retorno {Time, Value} en microsegundos. <https://www.erlang.org/docs/26/man/timer.html>

Vargas Rodríguez, A. y Ríos, J. P. Repositorio del proyecto y salida-programa.txt. <https://github.com/Reposao/parcial-elixir-mensajeria>

Programación III · Parcial 1

Alejandro Vargas Rodríguez y Juan Pablo Ríos
Carrera 15 Calle 12 Norte
Armenia, Quindío – Colombia
Universidad del Quindío · 2026



UNIQUINDÍO, en conexión territorial

Carrera 15 Calle 12 Norte Tel: (606) 7 35 93 00 Armenia - Quindío - Colombia

www.uniquindio.edu.co